Mount Kenya University

Unlocking Infinite Possibilities

Student Name	Eddy Kering
Admission Number	BSCCS/2024/61947
Course / Class	Computer Science
Unit Code	BIT4138
Unit Name	Advanced Cryptography
Lecturer Name	Michael Nyoro
Semester / Academic Year	2026

Project Title	BIT4138 Advanced Cryptography — Classical, Stream, Block and Asymmetric Cipher Implementations
Selected Technologies	Python 3, VS Code, Cryptography Library, PyCryptodome, OpenSSL, Git, GitHub
GitHub / Portfolio Link	https://github.com/keringyegon-byte/BIT4138-Cryptography

TABLE OF CONTENTS

Week 6: Symmetric Ciphers II — Substitution-Permutation Networks

SPN Design, S-Boxes, Non-Linearity and Avalanche Effect

Overview

This week focused on Substitution-Permutation Networks (SPNs), the cipher structure used in modern encryption standards such as AES. Unlike Feistel ciphers which operate on half-blocks, SPNs apply substitution and permutation across the entire block each round. Key concepts covered include S-Box design, non-linearity, the avalanche effect, and the differences between SPN and Feistel structures.

Key Concepts Applied

- SPN applies three operations per round: Substitution (S-Box), Permutation, and Key Mixing
- S-Boxes introduce non-linearity — making it impossible to describe the cipher with simple linear equations
- Permutation spreads (diffuses) data across the entire block each round
- The avalanche effect ensures a 1-bit change in input causes at least 50% of output bits to change
- AES is an SPN cipher; DES uses a Feistel structure

Feistel Network vs SPN — Comparison

Criteria	Feistel Network	SPN
Data Handling	Splits block into two halves	Operates on entire block at once
Operations	Round function applied to half	S-Box + Permutation on full block
Decryption	Same structure, reversed keys	Requires separate inverse operations
Example Cipher	DES	AES, PRESENT, SERPENT
Security	Good — depends on round function	Strong — high non-linearity from S-Boxes

Required Evidence

Fig 1: SPN Implementation Code

```python
def spn_encrypt(plaintext, key, rounds=3, verbose=True):
    sk = get_subkeys(key, rounds)
    s = plaintext
    if verbose: print(f"  Initial state    : {s:016b} ({s})")
    for r in range(rounds):
        s ^= sk[r]
        if verbose: print(f"  Round {r+1} key mix   : {s:016b}")
        s = substitute(s, SBOX)
        if verbose: print(f"  Round {r+1} sub       : {s:016b}")
        if r < rounds-1:
            s = permute(s, PBOX)
            if verbose: print(f"  Round {r+1} perm      : {s:016b}")
    s ^= sk[rounds]
    if verbose: print(f"  Ciphertext      : {s:016b} ({s})")
    return s
```

Fig 1: Python source code showing the SPN implementation with S-Box substitution, permutation, and key mixing functions

Fig 2: S-Box Substitution Output

```
Select option: 1

------------------------------------------------------
  S-Box Lookup Table
------------------------------------------------------

  Input    Binary     Output   Binary
  ---------------------------------------
  0        0000       14       1110
  1        0001       4        0100
  2        0010       13       1101
  3        0011       1        0001
  4        0100       2        0010
  5        0101       15       1111
  6        0110       11       1011
  7        0111       8        1000
  8        1000       3        0011
  9        1001       10       1010
  10       1010       6        0110
  11       1011       12       1100
  12       1100       5        0101
  13       1101       9        1001
  14       1110       0        0000
  15       1111       7        0111

  All outputs unique : YES - Secure
```

Fig 2: Terminal output showing the S-Box lookup table and substitution results for sample inputs

Fig 3: Permutation Demonstration

```
  [1] Show S-Box Lookup Table
  [2] Demonstrate Substitution
  [3] Demonstrate Permutation
  [4] Full SPN Encryption and Decryption
  [5] Avalanche Effect Test
  [6] Exit

Select option: 3

------------------------------------------------------
  Permutation Demo
------------------------------------------------------
  Enter 16-bit integer (e.g. 12345): 12345

  Original   : 0011000000111001  (12345)
  Permuted   : 0001000010101011  (4267)
  Restored   : 0011000000111001  (12345)
  Correct    : YES
```

Fig 3: Terminal output showing bit/character permutation being applied to a block and the resulting rearranged output

Fig 4: Full SPN Encryption and Decryption

```
Select option: 4

------------------------------------------------------
  Full SPN Encryption and Decryption
------------------------------------------------------
  Enter plaintext (0-65535, e.g. 9876): 87650

  Key    : 1010110011001010
  Rounds : 3

--- Encryption ---
  Initial state   : 0101011001100010 (22114)
  Round 1 key mix  : 1111101010101000
  Round 1 sub      : 0111011001100011
  Round 1 perm     : 0000111011111001
  Round 2 key mix  : 1011000000000111
  Round 2 sub      : 1100111011101000
  Round 2 perm     : 1111111001100000
  Round 3 key mix  : 0111011011000010
  Round 3 sub      : 1000101101011101
  Ciphertext      : 0001000100001011 (4363)

--- Decryption ---

  Original  : 22114
  Encrypted : 4363
  Decrypted : 22114
  Match     : YES - Correct!
```

Fig 4: Terminal output showing a plaintext message encrypted through multiple SPN rounds and successfully decrypted back to original

Fig 5: Avalanche Effect Test

```
  [1] Show S-Box Lookup Table
  [2] Demonstrate Substitution
  [3] Demonstrate Permutation
  [4] Full SPN Encryption and Decryption
  [5] Avalanche Effect Test
  [6] Exit

Select option: 5

------------------------------------------------------
  Avalanche Effect Test
------------------------------------------------------
  Enter plaintext (e.g. 9876): 43546

  Original  input  : 1010101000011010 (43546)
  Modified  input  : 1010101000011011 (43547)  [1 bit flipped]
  Original  cipher : 1001110001000010 (40002)
  Modified  cipher : 1011010001001100 (46156)
  Bits changed     : 5/16 (31.2%)
  Avalanche effect : MODERATE
```

Fig 5: Output comparing encryption of two near-identical inputs (e.g. HELLO vs HELLo), showing significantly different ciphertext outputs

Student Reflection

Implementing the SPN demonstrated how combining substitution and permutation across multiple rounds creates strong encryption. The S-Box introduces non-linearity that resists algebraic attacks, while permutation ensures diffusion across the full block. The avalanche effect test confirmed that changing even one character produces a completely different ciphertext, validating the SPN security design.

Weekly Summary

This week I implemented a Substitution-Permutation Network in Python, incorporating S-Box substitution, bit permutation, and XOR key mixing across multiple rounds. The comparison between Feistel and SPN structures clarified why AES adopts the SPN model. Avalanche effect testing and non-linearity analysis confirmed the security advantages of well-designed S-Boxes in modern block cipher design.